

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY - VNU



DISCRETE STRUCTURE (CO1007)

ASSIGNMENT

A* ALGORITHMS

Instructor(s): Nguyễn An Khương, CSE-HCMUT
Lê Hồng Trang, CSE-HCMUT
Trần Tuấn Anh, CSE-HCMUT
Phạm An Vinh, CSE-HCMUT
Mai Xuân Toàn, CSE-HCMUT
Trần Hồng Tài, CSE-HCMUT

Class: CC02

Student: Dương Ngọc Minh Thư

HO CHI MINH CITY, JUNE 2026

Table of Contents

1	Introduction	2
1.1	The shortest path problem	2
1.2	A* algorithm	2
2	Algorithm design	3
2.1	Overview of the A* Algorithm	3
2.2	Pseudo Code of the A* Algorithm	4
2.3	Data Structures	5
3	Requirement answer	6
3.1	Task 1 — Social Network	6
3.1.1	Explanation	6
3.1.2	Manual example	7
3.2	Task 2 — Drone Delivery	10
3.2.1	Explanation	10
3.2.2	Manual Example	11
3.3	Task 3 — Warehouse Robot	14
3.3.1	Manual Example	14
3.4	Task 4 — Evacuation Route	16
3.4.1	Conversion from Grid to Weight Matrix	16
3.4.2	Manual Example Run	18
4	Program usage	20
5	References	21

1 Introduction

1.1 The shortest path problem

The shortest path problem is famous in the field of computer science. To solve the shortest path problem means to find the shortest.

In the shortest path problem, a graph can represent anything from a road network to a communication network, where the vertices can be intersections, cities, or routers, and the edges can be roads, flight paths, or data links.

For example, the shortest path from vertex 0 to 4 in the graph below is 0->1->2->5->4, with a total path weight of $4 + 8 + 4 + 10 = 26$. Other paths from 0 to 4 are also possible, but they have a higher total weight, so they can not be considered to be the shortest path.

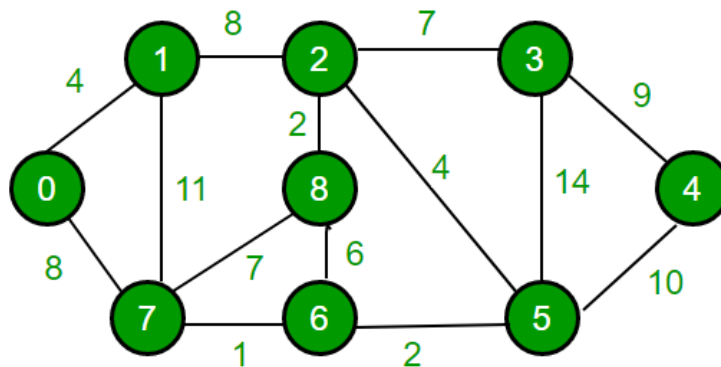


Figure 1: Example of shortest path in graph

1.2 A* algorithm

The A* algorithm is a powerful and widely used graph traversal and path finding algorithm. It finds the shortest path between a starting node and a goal node in a weighted graph.

A* combines the strengths of Dijkstra's algorithm and Greedy Best-First Search by using a cost function defined as:

$$f(n) = g(n) + h(n)$$



where:

- $g(n)$ is the actual cost from the start node to the current node.
- $h(n)$ is the heuristic estimate of the cost from the current node to the goal.
- $f(n)$ is the total estimated cost of the cheapest solution through node n .

In this report, the A* algorithm is applied to four different pathfinding problems: Degrees of Separation in a Social Network, Drone Delivery in 2D Space with Three Heuristics, Warehouse Robot Navigation with Obstacles, and Evacuation Route Planning.

2 Algorithm design

2.1 Overview of the A* Algorithm

The A* algorithm is a pathfinding algorithm used to determine the shortest path between a start node and a goal node.

The algorithm uses two main sets:

- **OPEN set:** contains nodes that are discovered but have not been explored yet.
- **CLOSED set:** contains nodes that have already been explored.

Initially, the OPEN set contains only the start node, while the CLOSED set is empty.

During each iteration, the algorithm selects the node with the smallest $f(n)$ value from the OPEN set. This node becomes the current node for expansion.

If the current node is the goal node, the algorithm reconstructs the path and terminates. Otherwise:

- The current node is removed from the OPEN set.
- The current node is added to the CLOSED set.
- All valid neighboring nodes are examined.

For each neighbor:

- If the neighbor is already in the CLOSED set, it is ignored.

- Otherwise, the algorithm calculates:

$$g(n') = g(n) + \text{movement cost}$$

- The heuristic value $h(n')$ is estimated using the selected heuristic function.
- The total cost is calculated as:

$$f(n') = g(n') + h(n')$$

- If a better path is found, the parent node and cost values are updated.

The parent of each node is stored so that the final path can be reconstructed after reaching the goal node.

2.2 Pseudo Code of the A* Algorithm

```
1  OPEN = {start}
2  CLOSED = {}
3
4
5  g(start) = 0
6  h(start) = heuristic(start, goal)
7  f(start) = g(start) + h(start)
8
9  while OPEN is not empty:
10
11      current = node in OPEN
12              with smallest f value
13
14      if current == goal:
15          reconstruct path
16          return path
17
18      remove current from OPEN
19      add current to CLOSED
20
21      for each neighbor of current:
22
```

```
23         if neighbor is invalid
24             or blocked
25             or in CLOSED:
26                 continue
27
28         tentative_g =
29         g(current) + movement_cost
30
31         if tentative_g < g(neighbor):
32
33             parent(neighbor) = current
34
35             g(neighbor) = tentative_g
36
37             h(neighbor) =
38             heuristic(neighbor, goal)
39
40             f(neighbor) =
41             g(neighbor) + h(neighbor)
42
43             if neighbor not in OPEN:
44                 add neighbor to OPEN
45
46     return no path
```

2.3 Data Structures

Several data structures are used in the implementation of the A* algorithm.

PathNode Structure

The PathNode structure is used to store information about each step in the final path.

- **name:** stores the node name or movement direction
- **g:** actual path cost from the start node
- **h:** heuristic estimate to the goal
- **f:** total estimated cost



- **next**: pointer to the next node in the linked list

Open Set

The OPEN set stores nodes that have been discovered but not fully explored yet.

In this implementation, the OPEN set is represented using a vector.

Closed Set

The CLOSED set stores nodes that have already been explored in order to avoid revisiting them.

Cost Arrays

Several arrays are used to store pathfinding information:

- $g[n]$: actual movement cost from the start node
- $h[n]$: heuristic estimate to the goal node
- $f[n]$: total estimated cost

Parent Array

The parent array stores the previous node of each visited node.

This information is used to reconstruct the final path after reaching the goal node.

3 Requirement answer

3.1 Task 1 — Social Network

Explain how $h(n)$ is calculated and provide a manual example run.

3.1.1 Explanation

First, a vector queue is created and the goal node is added to it. All heuristic values $h(n)$ are initialized to infinity to indicate that the nodes have not been visited yet. The heuristic value of the goal node is then assigned as $h(n)=0$.

The algorithm repeatedly takes the first node from the queue as the current node. Next, it iterates through the adjacency matrix to find all nodes connected to the current node. For each



connected node whose heuristic value is still infinity (unvisited), the heuristic value is updated by taking the heuristic value of the current node and adding 1. The newly updated node is then added to the queue for further exploration.

This process continues iteratively until the queue becomes empty, meaning that the minimum number of connections from every reachable node to the goal node has been determined.

3.1.2 Manual example

The following adjacency matrix is used:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Where:

- Person 0 is connected to Person 1 and Person 2
- Person 1 is connected to Person 3
- Person 2 is connected to Person 3 and Person 4
- Person 3 is connected to Person 4

Suppose:

$$\text{Start Person} = 0$$

$$\text{Goal Person} = 4$$

The algorithm starts from the goal node (Person 4).

Step 1: Initialize

Set the goal node as the first current node.

$$h(4) = 0$$



OpenSet before selecting current node:

[4]

Choose node 4 as the current node and remove it from the OpenSet.

OpenSet after removal:

[]

Step 2: Visit neighbors of 4

Person 2 and Person 3 are connected to Person 4.

$$h(2) = 1$$

$$h(3) = 1$$

Add nodes 2 and 3 into the OpenSet.

OpenSet:

[2, 3]

Choose node 2 as the current node and remove it from the OpenSet.

OpenSet after removal:

[3]

Step 3: Visit neighbors of 2

Person 0 is connected to Person 2.

$$h(0) = 2$$

Add node 0 into the OpenSet.

OpenSet:

[3, 0]

Choose node 3 as the current node and remove it from the OpenSet.

OpenSet after removal:

[0]



Step 4: Visit neighbors of 3

Person 1 is connected to Person 3.

$$h(1) = 2$$

Add node 1 into the OpenSet.

OpenSet:

$$[0, 1]$$

Choose node 0 as the current node and remove it from the OpenSet.

OpenSet after removal:

$$[1]$$

Step 5: Visit neighbors of 1

All neighbors of Person 1 already have heuristic values assigned, so no updates are needed.

OpenSet:

$$[1]$$

Choose node 1 as the current node and remove it from the OpenSet.

OpenSet after removal:

$$[]$$

Since the OpenSet is now empty, the algorithm terminates.

Final Heuristic Values

Person	$h(n)$
0	2
1	2
2	1
3	1
4	0

Thus, the heuristic function correctly estimates the minimum remaining number of connections

from each node to the goal node.

3.2 Task 2 — Drone Delivery

Compare the three heuristics with manual example runs where each heuristic has its advantage.

3.2.1 Explanation

Method 1 — Manhattan Distance

The Manhattan distance calculates the total horizontal and vertical distance between two points.

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

This heuristic works well when movement is mainly restricted to horizontal and vertical directions. Advantages:

- Simple and fast to compute
- Suitable for grid-based movement
- Produces stable estimates

Disadvantages:

- May overestimate actual movement when diagonal movement is allowed

Method 2 — Euclidean Distance

The Euclidean distance measures the straight-line distance between two points. This heuristic is commonly used in continuous 2D space problems.

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Advantages:

- Provides realistic geometric distance
- Usually gives more accurate estimates
- Suitable for free movement in any direction

Disadvantages:

- More computationally expensive due to square root calculations



Method 3 — Chebyshev Distance

The Chebyshev distance measures the maximum difference between the coordinates.

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Advantages:

- Efficient for diagonal navigation
- Faster computation than Euclidean distance
- Suitable for chessboard-like movement

Disadvantages:

- Less accurate in weighted graphs with varying edge costs

3.2.2 Manual Example

Suppose the coordinates are:

Point	(x, y)
0	$(0, 0)$
1	$(2, 1)$
2	$(4, 2)$
3	$(5, 5)$

Assume:

Goal Point = 3

Thus:

$$(x_2, y_2) = (5, 5)$$

Method 1 — Manhattan Distance

For Point 0:

$$h(0) = |0 - 5| + |0 - 5| = 10$$

For Point 1:



$$h(1) = |2 - 5| + |1 - 5| = 7$$

For Point 2:

$$h(2) = |4 - 5| + |2 - 5| = 4$$

Final heuristic values:

Point	$h(n)$
0	10
1	7
2	4
3	0

Method 2 — Euclidean Distance

For Point 0:

$$\begin{aligned}h(0) &= \sqrt{(0 - 5)^2 + (0 - 5)^2} \\&= \sqrt{25 + 25} \\&= \sqrt{50} \approx 7.07\end{aligned}$$

For Point 1:

$$\begin{aligned}h(1) &= \sqrt{(2 - 5)^2 + (1 - 5)^2} \\&= \sqrt{9 + 16} \\&= \sqrt{25} = 5\end{aligned}$$

For Point 2:

$$h(2) = \sqrt{(4 - 5)^2 + (2 - 5)^2}$$



$$= \sqrt{1 + 9}$$

$$= \sqrt{10} \approx 3.16$$

Final heuristic values:

Point	$h(n)$
0	7.07
1	5
2	3.16
3	0

Method 3 — Chebyshev Distance

For Point 0:

$$h(0) = \max(|0 - 5|, |0 - 5|) = 5$$

For Point 1:

$$h(1) = \max(|2 - 5|, |1 - 5|) = 4$$

For Point 2:

$$h(2) = \max(|4 - 5|, |2 - 5|) = 3$$

Final heuristic values:

Point	$h(n)$
0	5
1	4
2	3
3	0

From the example above:

- Manhattan distance produces the largest heuristic values because it only considers horizontal



and vertical movement.

- Euclidean distance provides the most realistic geometric estimate.
- Chebyshev distance gives smaller values and is suitable when diagonal movement is allowed.

3.3 Task 3 — Warehouse Robot

Provide a manual example run with a small warehouse grid, showing the chosen directions and the computed $g(n)$, $h(n)$, and $f(n)$ at each step and comparing both heuristic modes.

3.3.1 Manual Example

In this task, the warehouse is represented as a grid where:

- 0 represents a passable cell
- 1 represents an obstacle

The robot can move in 8 directions:

- Horizontal and vertical moves cost 1
- Diagonal moves cost 1.5

The following warehouse grid is used:

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Suppose:

Start Position = (0, 0)

Goal Position = (3, 3)

The cells (1, 1) and (1, 2) are blocked obstacles.



Method 1 — Manhattan Distance

The heuristic function is:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

The selected path is:

$$(0, 0) \rightarrow (1, 0) \rightarrow (2, 1) \rightarrow (3, 2) \rightarrow (3, 3)$$

The chosen directions are:

$$\text{Down} \rightarrow \text{Down-Right} \rightarrow \text{Down-Right} \rightarrow \text{Right}$$

Step-by-step Cost Calculation

Node	Direction	$g(n)$	$h(n)$	$f(n)$
(1, 0)	Down	1	5	6
(2, 1)	Down-Right	2.5	3	5.5
(3, 2)	Down-Right	4	1	5
(3, 3)	Right	5	0	5

Method 2 — Chebyshev Distance

The heuristic function is:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Using the same warehouse layout, the selected path is:

$$(0, 0) \rightarrow (1, 0) \rightarrow (2, 1) \rightarrow (3, 2) \rightarrow (3, 3)$$

The chosen directions are:

$$\text{Down} \rightarrow \text{Down-Right} \rightarrow \text{Down-Right} \rightarrow \text{Right}$$

Step-by-step Cost Calculation

Node	Direction	$g(n)$	$h(n)$	$f(n)$
(1, 0)	Down	1	3	4
(2, 1)	Down-Right	2.5	2	4.5
(3, 2)	Down-Right	4	1	5
(3, 3)	Right	5	0	5

Comparison Between the Two Heuristics

From the example above:

- Manhattan distance produces larger heuristic values because it only considers horizontal and vertical movement.
- Chebyshev distance produces smaller heuristic values and better reflects diagonal movement.
- Since diagonal movement is allowed in this problem, Chebyshev distance is generally more suitable for estimating the remaining cost.

3.4 Task 4 — Evacuation Route

Describe the conversion process from grid to weight matrix and provide a manual example run comparing both heuristic modes.

3.4.1 Conversion from Grid to Weight Matrix

In Task 4, the floor plan is initially represented as a 2D grid. Each cell in the grid is converted into a vertex in a graph, and the graph is stored using a weight matrix.

Grid Representation

- 0 represents a passable cell
- 1 represents an obstacle

The following floor plan is used:



$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The cell (1, 1) is blocked.

Vertex Conversion

Each grid cell is converted into a vertex using the following formula:

$$\text{vertex} = x \times n + y$$

Where:

- x is the row index
- y is the column index
- n is the number of columns

For this example:

Coordinate	Vertex ID
(0, 0)	0
(0, 1)	1
(0, 2)	2
(1, 0)	3
(1, 2)	5
(2, 0)	6
(2, 1)	7
(2, 2)	8

Building the Weight Matrix

For each passable cell:

- Horizontal and vertical neighbors are connected with cost 1
- Diagonal neighbors are connected with cost 1.5



- Obstacles are ignored

Example:

- Vertex 0 (0,0) connects to:
 - Vertex 1 (0,1) with cost 1
 - Vertex 3 (1,0) with cost 1
- Vertex 1 (0,1) connects to:
 - Vertex 0 with cost 1
 - Vertex 2 with cost 1
 - Vertex 5 with cost 1.5

Thus, the grid is successfully transformed into a weighted graph represented by the weight matrix.

3.4.2 Manual Example Run

Suppose:

Start Position = (0,0)

Exit Position = (2,2)

Method 1 — Manhattan Distance

The heuristic function is:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

The selected path is:

$$(0,0) \rightarrow (0,1) \rightarrow (1,2) \rightarrow (2,2)$$

Step-by-step Calculation

Node	$g(n)$	$h(n)$	$f(n)$
(0,0)	0	4	4
(0,1)	1	3	4
(1,2)	2.5	1	3.5
(2,2)	3.5	0	3.5



Method 2 — Chebyshev Distance

The heuristic function is:

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

The selected path is:

$$(0, 0) \rightarrow (0, 1) \rightarrow (1, 2) \rightarrow (2, 2)$$

Step-by-step Calculation

Node	$g(n)$	$h(n)$	$f(n)$
(0, 0)	0	2	2
(0, 1)	1	2	3
(1, 2)	2.5	1	3.5
(2, 2)	3.5	0	3.5

Comparison Between Heuristic Modes

From the example above:

- Manhattan distance produces larger heuristic values because it measures only horizontal and vertical distance.
- Chebyshev distance produces smaller estimates and better supports diagonal movement.
- Since diagonal movement is allowed in the evacuation problem, Chebyshev distance is generally more suitable for estimating the remaining path cost.



4 Program usage

How to Run the Program

All files should be placed in the same directory:

- `PathNode.h` and `PathNode.cpp`: contain the `PathNode` structure and related functions.
- `Algo.h` and `Algo.cpp`: contain the functions implementing the algorithms used to solve the problem.
- `main.cpp`: the main program file, used to demonstrate how the program works through sample input/output.

Compilation:

```
g++ -o main main.cpp PathNode.cpp Algo.cpp
```

Execution:

```
./main
```



5 References

- [1] DataCamp. (n.d.). *A* search algorithm tutorial*. DataCamp. <https://www.datacamp.com/tutorial/a-star-algorithm>
- [2] GeeksforGeeks. (n.d.). *A* search algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/a-search-algorithm/>
- [3] Patel, A. (n.d.). *Introduction to the A* algorithm*. Stanford University. <https://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html>